



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Tarea Práctica #2: Gráfica (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 18 de
septiembre de 2024

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA

NoThreadedAverageCalculator

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class NoThreadedAverageCalculator {
8     public static void main(String[] args) {
9         String fileName = "aleatorios.txt";
10        List<Integer> numbers = new ArrayList<>();
11
12        long startTime = System.currentTimeMillis();
13
14        // Leer el archivo y almacenar los números en una lista
15        try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
16            String line = br.readLine();
17            String[] numberStrings = line.split(",");
18            for (String numberString : numberStrings) {
19                numbers.add(Integer.parseInt(numberString));
20            }
21        } catch (IOException e) {
22            e.printStackTrace();
23        }
24
25        // Calcular la suma de todos los números
26        long sum = 0;
27        for (int num : numbers) {
28            sum += num;
29        }
30
31        // Calcular el promedio
32        double average = (double) sum / numbers.size();
33
34        System.out.println("El promedio total es: " + average);
35
36        long endTime = System.currentTimeMillis();
37        long executionTime = endTime - startTime;
38        System.out.println("Tiempo de ejecucion: " + executionTime + " milisegundos");
39    }
40 }
41
```

RandomNumberGenerator

```
1 import java.io.FileWriter;
2
3
4
5 public class RandomNumbersGenerator {
6     public static void main(String[] args) {
7         Random random = new Random();
8         int amountOfNumbers = 1000000;
9         String fileName = "aleatorios.txt";
10
11        try (FileWriter writer = new FileWriter(fileName)) {
12            for (int i = 0; i < amountOfNumbers; i++) {
13                int randomNumber = random.nextInt(1000);
14                writer.write(randomNumber + (i < amountOfNumbers - 1 ? "," : ""));
15            }
16            System.out.println("Archivo generado: " + fileName);
17        } catch (IOException e) {
18            e.printStackTrace();
19        }
20    }
21 }
22
```

ThreadedAverageCalculator

```
1 import org.jfree.chart.ChartFactory;
2 import org.jfree.chart.ChartPanel;
3 import org.jfree.chart.JFreeChart;
4 import org.jfree.data.category.DefaultCategoryDataset;
5 import javax.swing.JFrame;
6 import javax.swing.SwingUtilities;
7 import java.io.BufferedReader;
8 import java.io.FileReader;
9 import java.io.IOException;
10 import java.util.List;
11 import java.util.concurrent.Callable;
12 import java.util.concurrent.ExecutorService;
13 import java.util.concurrent.Executors;
14 import java.util.concurrent.Future;
15 import java.util.ArrayList;
16 import java.util.Scanner;
17
18 public class ThreadedAverageCalculator {
19     private static List<Integer> partitionCounts = new ArrayList<>();
20     private static List<Double> executionTimes = new ArrayList<>();
21
22     public static void main(String[] args) throws Exception {
23         String fileName = "aleatorios.txt";
24         List<Integer> numbers = new ArrayList<>();
25
26         // Lee número máximo de particiones del usuario
27         Scanner scanner = new Scanner(System.in);
28         System.out.print("Ingrese el numero maximo de particiones (hilos): ");
29         int maxPartitions = scanner.nextInt();
30         scanner.close();
31
32         // Lee los números aleatorios del archivo y los almacena
33         try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
34             String line = br.readLine();
35             String[] numberStrings = line.split(",");
36             for (String numberString : numberStrings) {
37                 numbers.add(Integer.parseInt(numberString));
38             }
39         } catch (IOException e) {
40             e.printStackTrace();
41         }
42     }
43 }
```

```

42
43     int totalNumbers = numbers.size();
44
45     DefaultCategoryDataset dataset = new DefaultCategoryDataset();
46
47     // Ejecuta el cálculo con particiones desde 1 hasta el número máximo
48     for (int partitionCount = 1; partitionCount <= maxPartitions; partitionCount++) {
49         System.out.println("\n=== Ejecutando con " + partitionCount + " particion(es) ===");
50         double executionTime = calculateAverageWithPartitions(numbers, partitionCount, totalNumbers);
51         dataset.addValue(executionTime, "Tiempo de Ejecución (ms)", Integer.toString(partitionCount));
52     }
53
54     // Muestra gráfica
55     showChart(dataset);
56 }
57
58 private static double calculateAverageWithPartitions(List<Integer> numbers, int partitionCount, int totalNumbers) throws InterruptedException {
59     int partitionSize = totalNumbers / partitionCount;
60     ExecutorService executor = Executors.newFixedThreadPool(partitionCount);
61     List<Future<Double>> results = new ArrayList<>();
62
63     long startTime = System.currentTimeMillis();
64
65     // Asigna segmentos de la lista a los hilos
66     for (int i = 0; i < partitionCount; i++) {
67         int start = i * partitionSize;
68         int end = (i == partitionCount - 1) ? totalNumbers : (i + 1) * partitionSize;
69         List<Integer> segment = numbers.subList(start, end);
70         Callable<Double> calculator = new AverageCalculator(segment);
71         Future<Double> result = executor.submit(calculator);
72         results.add(result);
73     }
74
75     double totalSum = 0;
76
77     try {
78         // Obtiene los resultados de los hilos y calcular el promedio total
79         for (Future<Double> result : results) {
80             totalSum += result.get() * partitionSize;
81         }
82
83         double overallAverage = totalSum / totalNumbers;
84         System.out.println("El promedio total es: " + overallAverage);

```

```

85
86     } catch (Exception e) {
87         e.printStackTrace();
88     } finally {
89         executor.shutdown();
90     }
91
92     long endTime = System.currentTimeMillis();
93     long executionTime = endTime - startTime;
94     System.out.println("Tiempo de ejecución: " + executionTime + " milisegundos");
95
96     return executionTime;
97 }
98
99 private static void showChart(DefaultCategoryDataset dataset) {
100     SwingUtilities.invokeLater(() -> {
101         JFrame frame = new JFrame("Tiempo de Ejecución con Diferentes Número de Particiones");
102         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
103
104         // Crear gráfico
105         JFreeChart chart = ChartFactory.createLineChart(
106             "Tiempo de Ejecución con Diferentes Número de Particiones",
107             "Número de Particiones",
108             "Tiempo de Ejecución (ms)",
109             dataset
110         );
111
112         // Crear y agregar panel de gráfico al marco
113         ChartPanel chartPanel = new ChartPanel(chart);
114         chartPanel.setPreferredSize(new java.awt.Dimension(800, 600));
115         frame.add(chartPanel);
116
117         // Ajustar el tamaño del marco y hacerlo visible
118         frame.pack();
119         frame.setVisible(true);
120     });
121 }
122 }
123
124 class AverageCalculator implements Callable<Double> {
125     private List<Integer> numbersToCompute;
126
127     public AverageCalculator(List<Integer> numbersToCompute) {
128
129     public AverageCalculator(List<Integer> numbersToCompute) {
130         this.numbersToCompute = numbersToCompute;
131     }
132
133     @Override
134     public Double call() {
135         double sum = 0;
136         for (int num : numbersToCompute) {
137             sum += num;
138         }
139         return sum / numbersToCompute.size();
140     }
141 }

```

RESULTADO

Ingrese el numero maximo de particiones (hilos): 12

=== Ejecutando con 1 particion(es) ===
El promedio total es: 499.394477
Tiempo de ejecucion: 16 milisegundos

=== Ejecutando con 2 particion(es) ===
El promedio total es: 499.394477
Tiempo de ejecucion: 11 milisegundos

=== Ejecutando con 3 particion(es) ===
El promedio total es: 499.3939773666862
Tiempo de ejecucion: 2 milisegundos

=== Ejecutando con 4 particion(es) ===
El promedio total es: 499.394477
Tiempo de ejecucion: 2 milisegundos

=== Ejecutando con 5 particion(es) ===
El promedio total es: 499.394477
Tiempo de ejecucion: 1 milisegundos

=== Ejecutando con 6 particion(es) ===
El promedio total es: 499.3924783626687
Tiempo de ejecucion: 1 milisegundos

=== Ejecutando con 7 particion(es) ===
El promedio total es: 499.3939774603173
Tiempo de ejecucion: 1 milisegundos

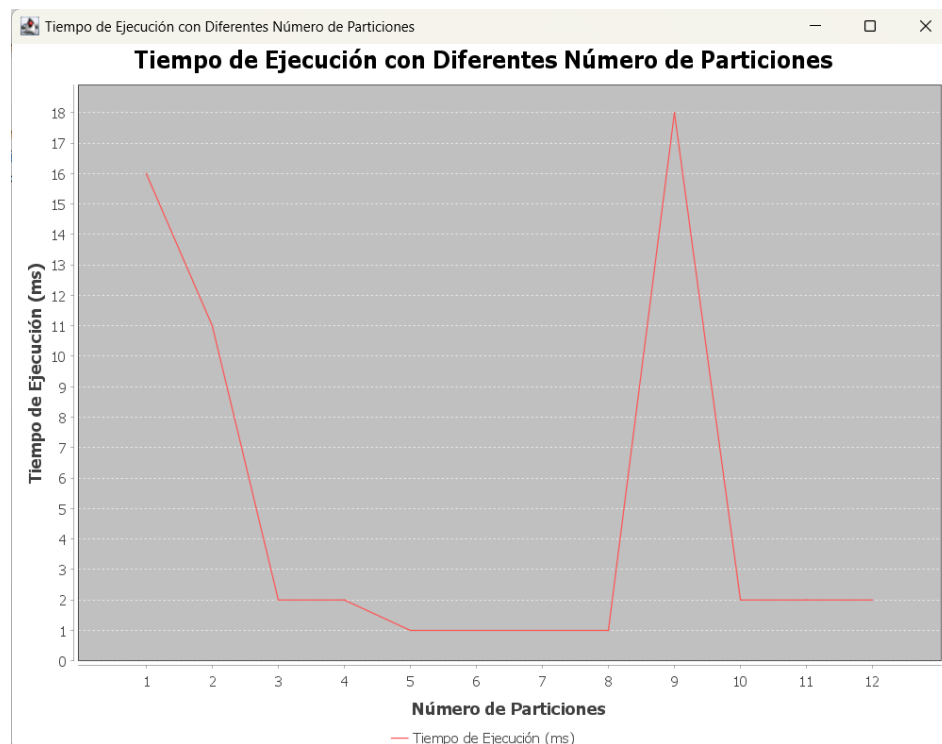
=== Ejecutando con 8 particion(es) ===
El promedio total es: 499.394477
Tiempo de ejecucion: 1 milisegundos

=== Ejecutando con 9 particion(es) ===
El promedio total es: 499.3939781103751
Tiempo de ejecucion: 18 milisegundos

=== Ejecutando con 10 particion(es) ===
El promedio total es: 499.394477
Tiempo de ejecucion: 2 milisegundos

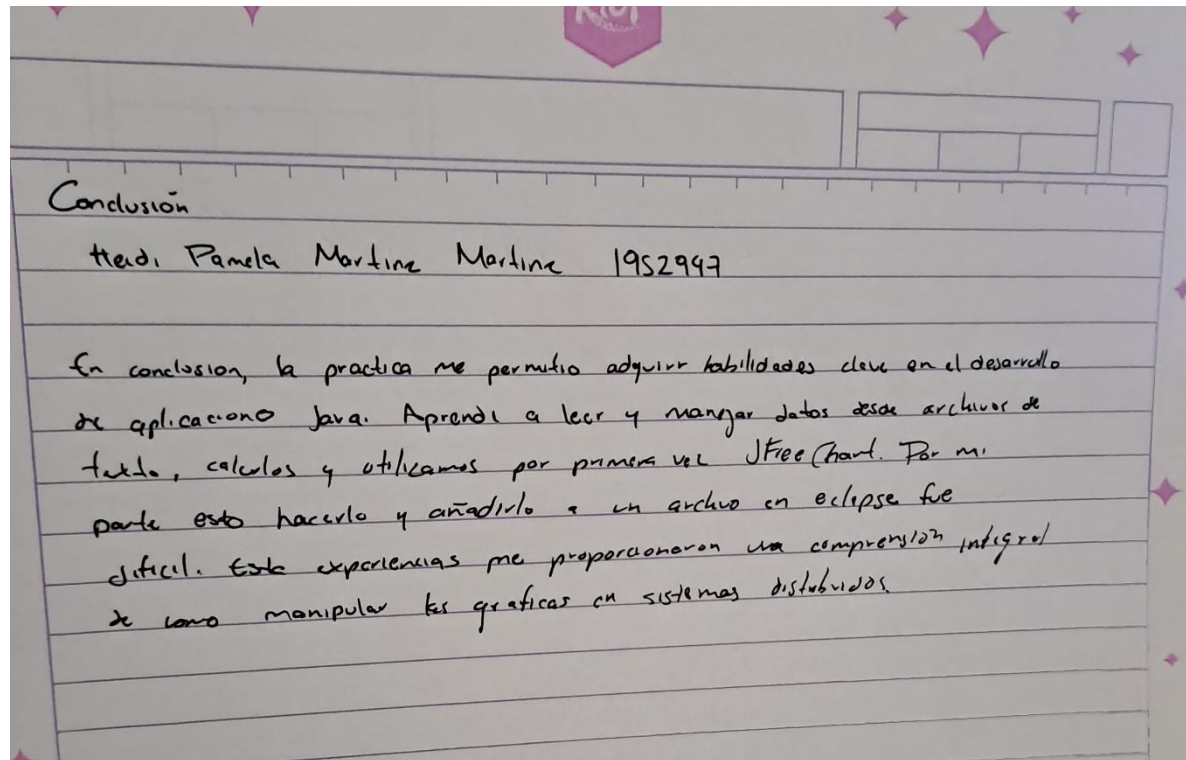
=== Ejecutando con 11 particion(es) ===
El promedio total es: 499.3939782576394
Tiempo de ejecucion: 2 milisegundos

=== Ejecutando con 12 particion(es) ===
El promedio total es: 499.39248216164486
Tiempo de ejecucion: 2 milisegundos



Conclusiones

1952947 - Heidi Pamela Martínez Martínez



Andrea Méndez 1845517

Las prácticas realizadas han demostrado claramente una evolución de la complementariedad de los conceptos a medida que avanzamos. En este caso el uso de herramientas para graficar ha sido esencial para poder observar y así comparar ambos tiempos de ejecución.

Alexia Hernandez Andrade 2109147

Conclusión Tarea Practica #2

En esta actividad podemos visualizar mejor los datos que recopilamos de la actividad pasada. Al graficar los tiempos se evidencia una variación significativa en los tiempos. Por lo tanto podemos observar nuevamente que es crucial encontrar un balance adecuado en la implementación de hilos para maximizar la eficiencia sin incurrir en obtener errores por el uso excesivo de concurrencia.

Alejandro Sauqué Pérez
Conclusión Individual Sistemas Distribuidas Práctica 2 Equipo

En conclusión me parece bastante interesante que estas prácticas han sido una secuencia que cada vez se complementan más conforme avanzamos. Es bueno hacer uso de herramientas de graficación ya que nos permite tener una referencia visual de las diferencias en los tiempos de ejecución con respecto al número de particiones. Es algo significativa a mi parecer que hay usualmente dos picos en los que los tiempos son menores, es decir, hay tendencia de que en esos dos puntos el tiempo de ejecución sea menor. Parece que hay más de un punto/número de particiones óptimo.

Conclusión

2077884

El objetivo de la práctica es observar y analizar el tiempo que toma la ejecución del programa (en nanosegundos), el tiempo que le toma a cada punto en la grafica, cada uno representando lo que tarda en ejecutarse de acuerdo a la cantidad de hilos usados en esa parte del proceso en específico. Esto permitiendonos ver una mejor manera de entender como es la ejecución de hilos en un programa.